Objectifs du chapitre :

- Connaitre l'objectif du module
- Définir : GL et logiciel
- Comprendre le cycle de vie d'un logiciel
- Connaitre les différents modèles de cycle de vie

I-1 Définitions

- Le terme Génie Logiciel (en anglais software engineering) désigne l'ensemble des méthodes, des techniques et outils concourant à la production d'un logiciel, au delà de la seule activité de programmation.

- Le génie logiciel est donc l'art de spécifier, de concevoir, de réaliser, et de faire évoluer, avec des moyens et dans des délais raisonnables, des programmes, des documentations et des procédures de qualité en vue d'utiliser un ordinateur pour résoudre certains problèmes.

I-2 Qu'est ce qu'un logiciel ?

Par logiciel on n'entend pas seulement l'ensemble des programmes informatiques (du code) associés à une application ou à un produit, mais également un certain nombre de documents se rapportant à ces programmes et nécessaires à leur installation, utilisation, développement et maintenance : spécification, schémas conceptuels, jeux de tests, mode d'emploi, etc.

Qualité exigée d'un logiciel

Si le génie logiciel est l'art de produire de bons logiciels, il est par conséquent nécessaire de fixer les critères de qualité d'un logiciel.

- **La validité** : C'est l'aptitude d'un produit logiciel à remplir exactement ses fonctions, définies par le cahier des charges et les spécifications.
- **La fiabilité** (ou robustesse) : C'est l'aptitude d'un produit logiciel à fonctionner dans des conditions anormales (quelque soit l'entrée par exemple).
- **L'extensibilité** : C'est la facilité avec laquelle un logiciel se prête à une modification ou à une extension des fonctions qui lui sont demandées.
- **La réutilisabilité** : C'est l'aptitude d'un logiciel à être réutilisé, en tout ou en partie, dans de nouvelles applications.
- **La compatibilité** : C'est la facilité avec laquelle un logiciel peut être combiné avec d'autres logiciels.
- **L'efficacité** : On dit d'un logiciel qu'il est efficace s'il utilise les ressources d'une manière optimale (comme la mémoire).
- **L'intégrité** : C'est l'aptitude d'un logiciel à protéger son code et ses données contre des accès non autorisé.
- **La facilité d'emploi** : Elle est liée à la facilité d'apprentissage, d'utilisation, d'interprétation des erreurs et de rattrapage en cas d'erreur d'utilisation.
- **La maintenabilité** : Elle correspond au degré de facilité de la maintenance d'un produit logiciel.

GL-3ING(IA/CS)

I-3 Cycle de vie d'un logiciel

I-3-1 Définition

Le cycle de vie du logiciel est la période de temps s'étalant du début à la fin du processus du logiciel. Il commence donc avec la proposition ou la décision de développer un logiciel et se termine avec sa mise hors service.
Le cycle de vie permet de détecter les erreurs au plus tôt et ainsi de maîtriser la qualité du produit, les délais de sa réalisation et les coûts associés.

I-3-2 Étapes du cycle de vie

Il existe de nombreux modèles de cycle de vie, les plus courants comportent les phases suivantes :

a) Étude d'opportunité (par des économistes en général)
b) Définition et analyse des besoins, spécification (par le commanditaire et des informaticiens)
c) Conception architecturale et élaboration des tests d'intégration
d) Conception détaillée et élaboration des tests unitaires
e) Codage (production du code source)
f) Tests unitaires et d'intégration
g) Implantation chez le commanditaire, essais avec les utilisateurs et validation
h) Formation des utilisateurs, utilisation, maintenance, évolution

Remarque : Ces étapes ne doivent pas être vues comme se succédant les unes aux autres de façon linéaire.
Il y a en général (toujours) des retours sur les phases précédentes, en particulier si les tests ne réussissent pas ou si les besoins évoluent.

a) Étude d'opportunité ou étude préalable

L'étude d'opportunité, aussi appelée étude préalable, est une phase cruciale dans le processus de développement d'un logiciel. Elle vise à déterminer si un projet est faisable, rentable et pertinent avant d'entamer le développement.

Objectif : Évaluer si le projet mérite d'être poursuivi en analysant les besoins, les risques, les coûts, les bénéfices, et les impacts potentiels. L'étude d'opportunité permet de prendre une décision éclairée sur le lancement ou non du projet.

Les composantes typiques de l'étude sont :

- **Analyse des besoins :** Identification des problèmes à résoudre et des attentes des utilisateurs.
- **Étude du marché :** Analyse de la concurrence et des alternatives existantes.
- **Évaluation des coûts et bénéfices**
- **Analyse des risques :** Identification des obstacles et des défis potentiels.
- **Impact sur l'organisation :** Étude des changements organisationnels requis.

GL-3ING(IA/CS)

Exemple : Création d'une application mobile pour une banque

b) Analyse (Spécification) : contrat entre client et fournisseur

Lors de la **phase d'analyse**, également appelée **phase de spécification** ou **spécification des exigences**, on analyse les besoins de l'utilisateur ou du système englobant et on définit ce que le logiciel devra faire : les **fonctionnalités, les contraintes et les critères de qualité**. Le résultat de la phase d'analyse est consigné dans un document appelé **cahier des charges du logiciel**.

Objectif : Traduire les besoins identifiés lors de l'analyse en spécifications détaillées et compréhensibles, tant pour les développeurs que pour les parties prenantes.

Types de spécifications

- **Spécifications fonctionnelles :** Ce que le logiciel doit faire.
  Exemples : fonctionnalités de l'application, interactions utilisateur, calculs spécifiques.
- **Spécifications non fonctionnelles :** Les contraintes de qualité, telles que les performances, la sécurité, la compatibilité, l'évolutivité.
- **Spécifications techniques :** Les aspects techniques liés à l'infrastructure, la base de données, les langages de programmation, etc.

Cahier des charges : Le cahier des charges sert de guide tout au long du projet pour s'assurer que le produit final correspond aux attentes. Il sert de **base contractuelle entre le client et le prestataire**, garantissant que le produit final correspond aux exigences établies.
Les besoins changent inévitablement. Le cahier des charges doit donc être conçu de manière à être facilement modifiable.

c) La conception du logiciel : contrat liant les membres de l'équipe

La phase d'analyse est suivie de la phase de conception, généralement décomposée en deux phases successives :

- **Conception générale (préliminaire)**

C'est l'étape où l'architecture globale du système est définie. Il s'agit de déterminer les grandes lignes de la solution sans entrer dans les détails de l'implémentation.

L'objectif de cette étape est d'Identifier les principales composantes du système, leur rôle et leurs interactions.

- Identifier les principales fonctionnalités nécessaires à l'application.
- Définir l'architecture globale de l'application.
- Prendre des décisions technologiques générales (plateformes, langages de programmation, etc.).
- Spécifier les interfaces entre les différents modules de l'application.

Le résultat de cette démarche est un document de conception générale.

GL-3ING(IA/CS)

- **Conception détaillée**

La conception détaillée affine la conception générale. Elle commence par décomposer les entités découvertes lors de la conception générale en entités plus élémentaires. Cette décomposition doit être poursuivie jusqu'au niveau où les entités sont faciles à implémenter et à tester, c'est-à-dire correspondent à des composants logiciels élémentaires.

On peut à cette étape utiliser le **langage UML (Unified Modeling Language)** pour la conception détaillée d'une application. UML est un langage de modélisation standardisé qui permet de représenter visuellement les différentes parties d'un système logiciel et leurs interactions. Il est particulièrement utile pour illustrer la structure et le comportement des modules, ainsi que les relations entre eux.

Avantages d'utiliser UML pour la conception détaillée

- **Visualisation claire** : Les diagrammes UML offrent une représentation visuelle des éléments du système et de leurs interactions, ce qui facilite la compréhension.
- **Communication facile** : Les diagrammes UML sont des standards que les équipes de développement, d'architecture et de gestion de projet peuvent facilement comprendre et utiliser pour communiquer.
- **Documentation structurée** : Les diagrammes peuvent servir de documentation du système, aidant à la maintenance et à l'évolution du logiciel.

<u>Qualité d'une conception</u>

La composante la plus importante de la qualité d'une conception est la maintenabilité. C'est en **maximisant la cohésion** à l'intérieur des composants et en **minimisant le couplage** entre ces composants que l'on parviendra à une conception maintenable.

Cohésion

La **cohésion** mesure dans quelle mesure les éléments d'un module ou d'une classe sont liés entre eux. Un module est dit **cohérent** lorsque ses composants travaillent ensemble pour réaliser une seule tâche ou un groupe de tâches étroitement liées. Plus la cohésion est élevée, plus le module est considéré comme de qualité.

Types de cohésion

1. **Cohésion fonctionnelle** : Tous les éléments d'un module sont nécessaires pour réaliser une seule tâche.
2. **Cohésion séquentielle** : Les éléments d'un module sont liés par des dépendances dans le flux de données.
3. **Cohésion communicative** : Les éléments d'un module partagent des données.

GL-3ING(IA/CS)

Couplage

Le **couplage** mesure le degré d'interdépendance entre les différents modules ou classes. Un système avec un **couplage faible** est considéré comme de meilleure qualité car les modules peuvent être modifiés, testés et maintenus indépendamment les uns des autres.

Types de couplage

1. **Couplage de contenu** : Un module accède directement aux données internes d'un autre module.
2. **Couplage commun** : Deux modules utilisent des données globales partagées.
3. **Couplage de contrôle** : Un module passe des informations de contrôle à un autre module.
4. **Couplage de données** : Un module utilise uniquement les données passées par un autre module.

GL- 3ING(IA/CS)